

Part A - Scientific proposal**A.1 Basic details****A.1.1 On-TRAQ: Transparently Retrofitting Applications for the Quantum Era****A.1.2 Abstract**

The transition to post-quantum cryptography presents a challenge that extends beyond replacing cryptographic algorithms: how can existing applications gain quantum-resistant communication with minimal changes? Existing solutions, such as virtual machines and security gateways, provide only partial answers to the challenge of transparently protecting application communication both within and across machines. We propose On-TRAQ, a systems approach to retrofitting quantum-resistant communication security into existing applications through interposition on system calls and shared memory accesses, without requiring application source-code modifications. System call interposition will mediate communication through OS interfaces, such as sockets, files, and pipes, while interposition on shared memory accesses will extend mediation to exchanges performed through direct memory accesses. Together, these mechanisms will support end-to-end protection across communicating application components. The research will establish the security guarantees achievable through this approach, investigate how to preserve application behaviour, and evaluate the performance costs of transparent protection. By separating communication security from application implementation, On-TRAQ aims to provide a practical migration path for existing software into the quantum era.

A.1.3 Summary

Future quantum computers will break critical cryptographic methods used today to protect digital communication. Preparing for this threat requires ways to bring new cryptographic techniques to existing software. One way to add these techniques is to rewrite the applications directly, but this is not always possible or practical: existing software is not always available with its original source code, and even when it is, rewriting can be difficult and costly. On the other hand, transparently retrofitting these techniques into existing software is challenging, and current solutions have several known limitations. On-TRAQ is an approach that will first identify all the points where applications exchange information, whether within or across computers. It will then apply, on the fly, the cryptographic operations needed to secure this information against attacks by quantum computers, without requiring changes to the applications' source code. In addition to the security guarantees of the solution, this project will also focus on usability and performance, since both are typically barriers to adoption. All in all, On-TRAQ aims to provide a viable solution for organizations that want to migrate their software into the quantum era.

A.1.4 Keywords

post-quantum cryptography; system call interposition; shared memory; transparent application retrofitting; end-to-end communication security

A.2 Scientific proposal

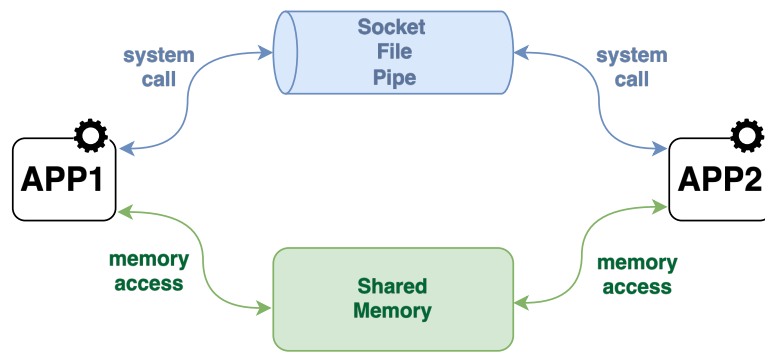


Figure 1: Applications can communicate through OS interfaces using system calls, or through shared memory using memory accesses. Pipes, files, and shared memory can only be used for communication on the same machine, while sockets can be used for communication both on the same machine and across different machines.

A.2.1 Research topic

The transition to quantum-resistant cryptography is an urgent European and Dutch priority, reflected in the EU's coordinated roadmap, the Dutch NCSC's guidance, and the joint AIVD, CWI, and TNO migration handbook [1, 2, 3]. Expert discussions of post-quantum security also emphasize the need to begin migration early and the risks of delayed adoption [4, 5]. The EU roadmap calls for Member States to begin transitioning by the end of 2026 and for critical infrastructure protection to migrate by the end of 2030 [1]. Preparing existing systems is particularly challenging: the AIVD highlights long-lived systems that are difficult or impossible to update and warns that migration can take many years [6], while ENISA identifies protocol-integration challenges that extend beyond replacing cryptographic algorithms [7], motivating practical ways to retrofit quantum-resistant protection.

Retrofitting this protection requires identifying every point at which applications exchange information. Applications can communicate using various methods: sockets, shared memory, pipes, and files, as illustrated in Figure 1. For communication across different machines, applications use sockets, which create a communication channel between the different hosts; both ends of the communication then use system calls, requests that an application makes to the operating system kernel, to send and receive data through this channel. Once applications reside on the same physical machine, they can also use sockets, or other system call-based techniques such as pipes and files, to communicate with each other. However, shared memory is the dominant communication method on the same machine, owing to its efficiency: it only requires memory accesses, whereas system call-based techniques are used only as a backup or by legacy applications [8].

Rewriting applications to support quantum-resistant communication is, of course, an option, but it is not always feasible. Legacy software is often only available as a compiled executable, without access to its source code, which rules out this option entirely; government reports in both the United States and the Netherlands describe how such legacy systems, some running decades-old languages such as COBOL on unsupported hardware and software, remain in critical use across public administration and are difficult or costly to replace [9, 10]. Even when the source code is available, modifying it is time-consuming and error-prone, as changes to long-lived, poorly documented codebases can introduce subtle regressions. These constraints motivate solutions that can facilitate the migration to quantum-resistant communication without requiring applications to be rewritten.

Existing solutions provide only partial answers to this challenge. Security gateways act as intermediaries that wrap an application's existing network connections in a protective tunnel, for instance using TLS, without requiring any changes to the application itself [11, 12]. However, they do not encrypt the segment between the application and the gateway, in either direction, since this segment is typically assumed to run over a trusted local network, and, operating purely at the network level, they do not cover local communication on the same machine, such as shared memory, pipes, and files. IPsec protects network traffic at the IP layer by authenticating and encrypting packets exchanged between hosts [13], but its inherent complexity has limited interoperability across implementations [14], and it requires a compatible implementation and sufficient configuration privileges at both communicating endpoints, which is not available everywhere and makes adoption challenging in practice; like security gateways, it also does not cover local communication, including shared memory, pipes, and files. Finally, VM-based solutions run applications inside confidential virtual machines and can, in principle, protect both local communication, including shared memory

within the virtual machine boundary, and network communication, for example by automatically wrapping traffic in mutual TLS [15, 16]. However, such solutions typically require specialized confidential-computing hardware or changes to how applications are deployed, and the added virtualization, attestation, and encryption layers introduce significant performance overhead.

On-TRAQ's central idea is to build a security layer that interposes all communication between applications, regardless of whether the communicating applications reside on the same machine or on different machines. The key observation is that, regardless of the specific communication method used, data ultimately passes through memory: system calls exchange data through memory buffers supplied by the application, for instance when sending or receiving over a socket, pipe, or file, while shared memory communication accesses a memory region directly. On-TRAQ therefore proposes to interpose both on system calls and on accesses to shared memory, and to insert a post-quantum protection layer at these interposition points. Data leaving an application is then transparently encrypted before being written to the corresponding channel, and data arriving at an application is transparently decrypted before being delivered to it, without requiring changes to the communicating applications themselves.

Existing interposition solutions come with limitations. In this paragraph, we discuss the main approaches and their trade-offs. Interposition techniques, whether applied to system calls or to memory accesses, have traditionally been implemented through cross-process designs [17, 18, 19], in which a separate tracer process observes and mediates a target application. Such designs are secure, since process isolation prevents a potentially compromised application from tampering with the interposition logic; however, the frequent context switches required to hand control to the tracer on every intercepted operation introduce substantial performance overhead [8, 20]. In-process designs avoid this overhead by executing the interposition logic within the application's own address space, typically using binary rewriting. However, such approaches depend on brittle heuristics [21, 22] or representative inputs [23], and, because the interposition logic runs in the same address space as the application, a compromised application can, in principle, tamper with or bypass it entirely. Other approaches instead rely on intrusive modifications such as changes to the operating system or the underlying hardware [24, 25, 26], or custom libOS/libc environments [27]; while such modifications can improve performance or strengthen security, they do so at the cost of increased deployment complexity, reduced portability, and an expanded trusted computing base (TCB).

On-TRAQ's aim is to build a practical approach for secure and efficient retrofitting. We focus closely on potential usability, security, and performance side-effects, since such issues usually make the adoption of solutions harder. Some performance degradation from the post-quantum cryptographic operations themselves is inevitable and expected; On-TRAQ's goal is to ensure that the interposition mechanism itself does not introduce additional overhead beyond this unavoidable cost.

On-TRAQ is therefore guided by a central research question:

"Can we transparently retrofit quantum-resistant communication security into existing applications using an interposition-based approach?"

A.2.2 Approach

Hardware and binary rewriting agnostic ...
 table with drawbacks of other solutions and ours ...
 the comments below may be for the approach but we will see
 na valeis se kedriki thesi to diagramma pou na min afinei amfivolia gia to pou tha einai i lysi sou
 Hybrid blah blah approach ... for migration according to Kostas
 Start of initialization of connection or shared memory we make the new layer ...

A.2.3 Justification

A.2.4 Embedding of research and relevant expertise of researchers involved

Expertise
 Collaborators
 Infrastructure

A.2.5 Risk assessment

A.3 Scientific and/or societal impact

Applicants in the Open Competition Domain Science – M can choose to put primary focus on scientific impact, societal impact, or a combination thereof. This proposal focusses on

A.4 Literature/references (in this proposal)

- [1] European Commission. EU reinforces its cybersecurity with post-quantum cryptography, June 2025.
<https://digital-strategy.ec.europa.eu/en/news/eu-reinforces-its-cybersecurity-post-quantum-cryptography>. 23 June 2025.
- [2] Nationaal Cyber Security Centrum. Wat is quantumveilige cryptografie?
<https://www.ncsc.nl/quantumveilige-cryptografie/wat-is-quantumveilige-cryptografie>. Accessed September 2026.
- [3] AIVD, CWI, and TNO. The PQC migration handbook: Guidelines for migrating to post-quantum cryptography. Technical report, AIVD, CWI, and TNO, 2024.
<https://english.aivd.nl/documents/2024/12/3/the-pqc-migration-handbook>. Second edition.
- [4] Bart Preneel. Quantum safe migration: Technology and policy overview. Presentation at the PQCSA Workshop on Tackling the Quantum Threat, September 2025.
https://www.esat.kuleuven.be/cosic/events/standardization-pqc-fundamentals/wp-content/uploads/sites/13/2025/09/BP_postquantum_12sep_2025_v1.pdf. 12 September 2025.
- [5] Bart Preneel. Post-quantum cryptography (PQC): The risk of being late. Lecture at SecAppDev 2026, June 2026. <https://secappdev.org/2026/sessions/post-quantum-cryptography-pqc-the-risk-of-being-late/>. 1 June 2026.
- [6] Algemene Inlichtingen- en Veiligheidsdienst. Bescherm je nu tegen de dreiging van quantumcomputers. Technical report, Ministerie van Binnenlandse Zaken en Koninkrijksrelaties, April 2026.
<https://www.aivd.nl/site/binaries/site-content/collections/documents/2026/03/27/bescherm-je-nu-tegen-de-dreiging-van-quantumcomputer-2026/bescherm-je-nu-tegen-de-dreiging-van-quantumcomputers-2026.pdf>.
- [7] European Union Agency for Cybersecurity. Post-quantum cryptography: Integration study. Technical report, ENISA, October 2022.
<https://www.enisa.europa.eu/publications/post-quantum-cryptography-integration-study>.
- [8] Stijn Volckaert, Bart Coppens, Alexios Voulimeneas, Andrei Homescu, Per Larsen, Bjorn De Sutter, and Michael Franz. Secure and efficient application monitoring and replication. In *Proceedings of the 2016 USENIX Conference on Usenix Annual Technical Conference*, USENIX ATC '16, pages 167–179, Denver, CO, 2016. USENIX Association. ISBN 9781931971300.
<https://www.usenix.org/conference/atc16/technical-sessions/presentation/volckaert>.
- [9] U.S. Government Accountability Office. Information technology: Agencies need to plan for modernizing critical decades-old legacy systems. Technical report, GAO, 2025.
<https://www.gao.gov/products/gao-25-107795>. GAO-25-107795.
- [10] Adviescollege ICT-toetsing. Problematische legacy: Voorkomen is beter dan genezen. Technical report, Adviescollege ICT-toetsing (AclCT), March 2025. <https://www.adviescollegeicttoetsing.nl/documenten/2025/03/19/acict-handreiking-over-legacy>. 19 March 2025.
- [11] Zakir Durumeric, Zane Ma, Drew Springall, Richard Barnes, Nick Sullivan, Elie Bursztein, Michael Bailey, J. Alex Halderman, and Vern Paxson. The security impact of HTTPS interception. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, San Diego, CA, USA, 2017. doi: 10.14722/ndss.2017.23456. <https://www.ndss-symposium.org/ndss2017/ndss-2017-programme/security-impact-https-interception/>.
- [12] stunnel project. *stunnel(8): TLS Offloading and Load-Balancing Proxy*, 2026.
<https://www.stunnel.org/manual.html>. Manual page, accessed September 2026.
- [13] Stephen Kent and Karen Seo. Security architecture for the internet protocol. RFC 4301, December 2005.
<https://www.rfc-editor.org/info/rfc4301>.
- [14] Ivan Pepelnjak. Why is IPsec so complex? ipSpace.net blog, October 2013.
<https://blog.ipspace.net/2013/10/why-is-ipsec-so-complex/>.

- [15] Masanori Misono, Dimitrios Stavrakakis, Nuno Santos, and Pramod Bhatotia. Confidential VMs explained: An empirical analysis of AMD SEV-SNP and Intel TDX. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 8(3):36:1–36:42, 2024. doi: 10.1145/3700418.
- [16] Edgeless Systems. Contrast: Architecture overview, 2026. <https://docs.edgeless.systems/contrast/architecture/overview>. Accessed September 2026.
- [17] Stijn Volckaert, Bjorn De Sutter, Tim De Baets, and Koen De Bosschere. GHUMVEE: Efficient, effective, and flexible replication. In *Proceedings of the 5th International Symposium on Foundations and Practice of Security (FPS)*, pages 261–277, 2013.
- [18] *strace – Trace System Calls and Signals*, 2026. <https://man7.org/linux/man-pages/man1/strace.1.html>. Manual page, accessed September 2026.
- [19] GNU Project. *gdb – The GNU Debugger*, 2026. <https://man7.org/linux/man-pages/man1/gdb.1.html>. Manual page, accessed September 2026.
- [20] Emma Connor, Tyler McDaniel, Jared M. Smith, and Max Schuchard. PKU pitfalls: Attacks on PKU-based memory isolation systems. In *Proceedings of the USENIX Security Symposium*, 2020.
- [21] Kenichi Yasukata, Hajime Tazaki, Pierre-Louis Aublin, and Kenta Ishiguro. zpoline: A system call hook mechanism based on binary rewriting. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, pages 293–300, 2023.
- [22] Adriaan Jacobs, Merve Gülmez, Alicia Andries, Stijn Volckaert, and Alexios Voulimeneas. System call interposition without compromise. In *Proceedings of the IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2024.
- [23] Jesús María Gómez, Vissarion Moutafis, Antreas Dionysiou, Fernando Kuipers, Georgios Smaragdakis, Bart Coppens, and Alexios Voulimeneas. Clair obscur: The light and shadow of syscall interposition: From pitfalls to solutions with K23. In *Proceedings of the ACM/IFIP International Middleware Conference*, 2025.
- [24] David Schrammel, Samuel Weiser, Richard Sadek, and Stefan Mangard. Jenny: Securing syscalls for PKU-based memory isolation systems. In *Proceedings of the USENIX Security Symposium*, 2022.
- [25] David Schrammel, Samuel Weiser, Stefan Steinegger, Martin Schwarzl, and Daniel Gruss. Donky: Domain keys – efficient in-process isolation for RISC-V and x86. In *Proceedings of the USENIX Security Symposium*, 2020.
- [26] Mohammad Hedayati, Spyridoula Gravani, Ethan Johnson, John Criswell, Michael L. Scott, Kai Shen, and Mike Marty. Hodor: Intra-process isolation for high-throughput data plane libraries. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, 2019.
- [27] Fangfei Yang, Bumjin Im, Weijie Huang, Kelly Kaoudis, Anjo Vahldiek-Oberwagner, Chia-Che Tsai, and Nathan Dautenhahn. Endokernel: A thread-safe monitor for lightweight subprocess isolation. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, 2024.

Part B - Applicant and additional information

B.1 NWO Scientific Classifications (disciplines)

Discipline 1: _____ Weight: _____

B.2 Main applicant - additional information

Name (titles, initials, first name, last name): _____

Affiliation - university/institute: _____

Affiliation - department: _____

Position: _____

Paid position: _____

Type of contract: _____

End date of contract: _____

B.3 Other pending or future grant applications on the same research topic

Yes/no _____

Title proposal: _____

Applicant(s): _____

Funding agency and call: _____

Budget applied for: _____

Date of submission: _____

Estimated date of decision: _____

Difference with this proposal: _____

Describe difference: _____

B.4 Public summaries

Dutch title and summary _____

English title and summary _____

B.5 Questions Data management section

1. Will this project involve re-using existing research data?
 - ☐ Yes: From my own or a collaborator's prior research.
 - ☐ Yes: Publicly available data.
 - ☐ No: Have you considered re-using existing data but discarded the possibility? Why? _____
2. Will data be collected or generated that are suitable for reuse?
 - ☐ Yes: Please answer questions 3 and 4.
 - ☐ No: Please explain why the research will not result in reusable data or in data that cannot be stored or data that for other reasons are not relevant for reuse. _____
3. After the project has been completed, how will the data be stored for the long-term and made available for the use by third parties? Are there possible restrictions to data sharing or embargo reasons? Please state these here. _____
4. Will any costs (financial and time) related to data management and sharing/preservation be incurred?
 - ☐ Yes: Then please be sure to specify the associated expenses as 'implementation cost' within the budget post 'Material costs' in the budget table of this proposal.
 - ☐ No: All the necessary resources (financial and time) to store and prepare data for sharing/preservation are or will be available at no extra cost.